

Python: Object Oriented Programming

D. Leeuw

25 november 2025

0.0.0

© 2025 Dennis Leeuw



Dit werk is uitgegeven onder de Creative Commons BY-NC-SA Licentie en laat anderen toe het werk te kopiëren, distribueren, vertonen, op te voeren, en om afgeleid materiaal te maken, zolang de auteurs en uitgever worden vermeld als maker van het werk, het werk niet commercieel gebruikt wordt en afgeleide werken onder identieke voorwaarden worden verspreid.

1 Objects en Classes (OOP introductie)

Stel dat we een formule 1 racespel willen maken. We hebben voor dat spel een aantal auto's nodig die een aantal rondjes over een circuit racen elke auto moet een coureur hebben. We zouden een script kunnen maken per auto en aan dat script het merk van de automodel, het team waarvoor gereden wordt, en de coureur mee kunnen geven. Die scripts kunnen we afzonderlijk aanroepen, maar daarmee hebben we nog geen race-programma. De auto's rijden niet op hetzelfde circuit en ze kunnen ongelijk gestart worden. We zouden ook een functie kunnen maken die we auto noemen en deze elke keer aanroepen met de gewenste eigenschappen. Ook dat lijkt nog niet echt op de werkelijkheid. Wat we zouden willen is dat er een auto template is van waaruit we alle auto's kunnen maken.

Met object georiënteerd programmeren kunnen we dat maken. In OOP (Object Oriented Programming) programmeer je een Class. Een Class is een template waarin je beschrijft wat de eigenschappen van de auto zijn en de functies die de auto kan uitvoeren.

Als het programma draait kiezen de spelers hun auto, team en coureur en maak je van de Class een Object door bepaalde waarden te zetten. We zeggen dan dat we een Instance van een Class hebben. Het Object is een specifiek object met bepaalde waarden (variabelen) gezet die het “uniek” maken ten opzichte van de template. Deze waarden die gezet kunnen worden noemen we Instance Variables. Door voor elke speler zijn eigen instance van de class te maken hebben we dus voor elke speler zijn eigen object gebaseerd op zijn eigen instance variabelen.

Tot slot willen we dat de auto een aantal rondjes gaat rijden op een circuit. Hiervoor hebben we Methodes. Methodes zijn functies die het object kan uitvoeren. We zouden een Methode kunnen hebben die zegt dat een auto een rondje moet rijden. Aan deze method kunnen we een waarde meegeven, bijvoorbeeld 64. De auto dan 64 rondjes rijden. Er wordt dus een opdracht uitgevoerd.

Er zijn ook zaken die voor elke auto hetzelfde zijn, omdat dat de regels zijn die bij formule 1 horen. Zo heeft elke auto 4 wielen, 1 stuur, is maximaal 200 cm breed en de hoogte van de auto is 950 cm gemeten vanaf de bodemplaat. Dit zijn waarden die voor elke auto hetzelfde zijn en kunnen dus niet gewijzigd worden. Dit zijn de zogenaamde Class Variables.

De Class voor dit object zouden we zo kunnen beschrijven:

Class name racecar

Instance variables coureur, team, automodel

Class variables wielen=4, stuur=1, auto_breedte=200, auto_hoogte=950

Methode(s) rondjes(aantal)

1.1 Opdrachten

1. Voor een emulatie programma willen we een virtuele server kunnen maken. De server hangt in een (virtueel) 19-inch rek en is aangesloten op een (virtuele) 1G ethernet switch. Jij moet een Class voor de server verzinnen, beantwoord de volgende vragen voor deze Class:
 - (a) Wat zijn mogelijke Instance Variabelen (noem er 4)?
 - (b) Wat zijn de Class Variables (noem er minimaal 2)?
 - (c) Welke Methode(s) zou je aanbieden?
2. Een hostingbedrijf heeft je op hulp gevraagd. Zij willen hun klanten een extra dienst aanbieden. Alle web-sites die zij hosten voor hun klanten willen ze monitoren en zodra er een probleem is moet er een e-mail naar de klant gaan. Aan jou de taak om een Class te bedenken die een webserver zou kunnen monitoren.
 - (a) Wat zijn mogelijke Instance Variables (noem er minimaal 2)?
 - (b) Wat zijn mogelijke Class Variables (noem er minimaal 2)?
 - (c) Welke Methode(s) zou je aanbieden (noem er minimaal 2)?

Index

Class, 2

Class Variables, 2

Instance, 2

Instance Variables, 2

Methodes, 2

Object, 2

Object Oriented Programming, 2

OOP, 2